ADS I Class Project **Summer 2026**
Hardware Design with Chisel
Prof. Dr.-Ing. Wolfgang Kunz RPTU Kaiserslautern-Landau
M.Sc. Tobias Jauch Fachbereich Elektrotechnik und Informationstechnik
Andro Mazmishvili Entwurf informationstechnischer Systeme
Juriaan Muijsson
Tristan Schmitt

Preparation

Before you start with the implementation, please answer the following questions:

1. What RV32I ISA instructions use which of the described ALU operations?
2. What should happen, if the ALU receives an input other than the specified operations?
3. What corner cases should a testbench definitely cover for the described implementation?

Solutions / Expected Answers

What RV32I ISA instructions use which of the described ALU operations?

- ADD: add, addi
- SUB: sub
- AND: and, andi
- OR: or, ori
- XOR: xor, xori
- SLL: sll, slli
- SRL: srl, srli
- SRA: sra, srai
- SLT: slt, slti
- SLTU: sltu, sltiu
- PASSB: lui, auipc

What should happen, if the ALU receives an input other than the specified operations?
The ALU should ideally handle invalid operations with a default case that covers all inputs that are not explicitly defined. The default case should either produce a defined output (e.g., zero or a specific error code) or raise an exception or error signal.

What corner cases should a testbench definitely cover for the described implementation?

- Addition overflow: Test cases where the sum of two large positive numbers exceeds the maximum representable value for a 32-bit signed integer.
- Subtraction underflow: Test cases where subtracting a larger number from a smaller one results in a negative value.
- Bitwise operations: Test cases with all bits set to 0 and all bits set to 1 for AND, OR, and XOR operations.
- Shift operations: Test cases with shift amounts of 0, 31, and values greater than 31 to ensure correct behavior.
- Comparison operations: Test cases where operands are equal, one is greater than the other, and edge cases involving the maximum and minimum representable values.

Prof. Dr.-Ing. Wolfgang Kunz
M.Sc. Tobias Jauch
Andro Mazmishvili
Juriaan Muijsson
Tristan Schmitt

RPTU Kaiserslautern-Landau
Fachbereich Elektrotechnik und Informationstechnik
Entwurf informationstechnischer Systeme

- PASSB operation: Test cases to verify that operandB is correctly forwarded to the output without modification.
- Invalid operation codes: Test cases that provide operation codes outside the defined range to ensure the ALU handles them gracefully.

Coverage in UVM Testbenches

In UVM, functional coverage is captured with SystemVerilog `covergroups` that define `coverpoints` for stimulus and DUT responses (e.g., ALU operation type, operand value ranges, shift amounts) and crosses for meaningful combinations. Covergroups are typically sampled in monitors or scoreboards to observe what actually occurred on the interface, independent of the driven stimulus. Code coverage (line/toggle/FSM) is collected by the simulator and complements functional coverage to show exercised RTL. During simulation, coverage data is accumulated and written to a coverage database; after runs, use the simulator's reporting utilities to identify uncovered bins and plan additional constrained-random sequences or directed tests to close gaps.

```
1  // Coverage for operand ranges - categorize values into meaningful bins
2  operandA_cg: coverpoint vif.operandA {
3    bins zero           = {32'h00000000};
4    bins max_value      = {32'hFFFFFFFF};
5    bins max_positive   = {32'h7FFFFFFF};
6    bins min_negative   = {32'h80000000};
7    bins low_range      = {[32'h00000001 : 32'h0000FFFF]};
8    bins high_range     = {[32'hFFFF0000 : 32'hFFFFFFFE]};
9    bins mid_range      = {[32'h00010000 : 32'hFFFEFFFF]};
10 }
11
12 // Cross-coverage: Operation with operand characteristics
13 op_operandA_cross: cross operation_cg, operandA_cg {
14   // Interesting cross combinations
15   bins shift_with_zero_operand = binsof(operation_cg.SLL_op) && binsof(operandA_cg.zero);
16   bins compare_equal_operands = binsof(operation_cg.SLT_op) && binsof(operandA_cg.zero);
17   \dots
18   \dots
19 }
20
21   // Result coverage - track outputs
22 result_cg: coverpoint vif.aluResult {
23   bins result_zero        = {32'h00000000};
24   bins result_max         = {32'hFFFFFFFF};
25   bins result_max_pos     = {32'h7FFFFFFF};
26   bins result_min_neg     = {32'h80000000};
27   bins result_one         = {32'h00000001};
28   bins result_other       = default;
29 }
30
31 // Special coverage for comparison operations
32 comparison_result_cg: coverpoint vif.aluResult iff (vif.operation inside {SLT, SLTU}) {
33   bins comparison_true  = {32'h00000001};
34   bins comparison_false = {32'h00000000};
35 }
```